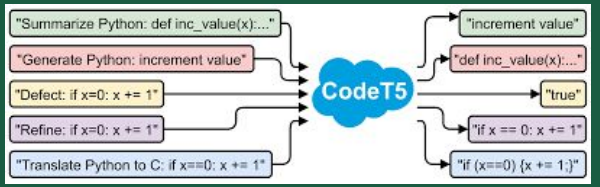
Guiding LLM-Based Vulnerability Repair using Structured Code Analysis Signals



LLaMA by Meta

deepseek

Qwen

Jia Sing, Raphael, Atsadet, Celest, Jin (50.021 Group 1)

THE PROBLEM:

Automatic Program Repair (APR) of *security vulnerabilities* in low-level languages such as C is *challenging* and relatively *underexplored*.

Existing Work in APR

NEURAL/LLM REPAIR

Recent work explores using neural models and large language models to automatically generate patches for buggy code. These systems learn repair patterns from historical bug-fix commits and generate candidate patches for vulnerable code [1,4].

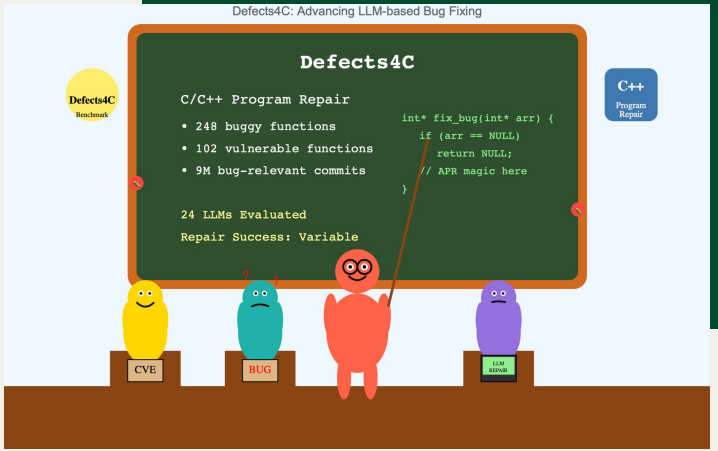
ANALYSIS-GUIDED REPAIR

Prior work shows that repair systems benefit from explicit program analysis signals. Techniques such as *vulnerability localisation*, *vulnerability classification*, and *repair ingredient retrieval* help guide patch generation toward correct and secure fixes [2,4].

C VULNERABILITY BENCHMARKS

APR evaluation on real-world C vulnerabilities remains limited. Defects4C provides realistic vulnerabilities with compilable code and test validation [3].

Defects4C



REAL VULNERABILITIES

Real-world C *single-function* security vulnerabilities paired with human-written fixes (n=102).

EXECUTABLE VALIDATION

Generated patches are evaluated using compilation and test-suite execution.

LLM BENCHMARK

Designed to evaluate APR systems using compilable code and executable tests.

Research Questions (RQs)

Hypothesis: Structured vulnerability analysis signals improve LLM-based repair performance.

RESEARCH QUESTION 1 (RQ1)

How effectively can LLM-based automated program repair systems fix *real-world C vulnerabilities in the Defects4C_vul benchmark*?

RESEARCH QUESTION 2 (RQ2)

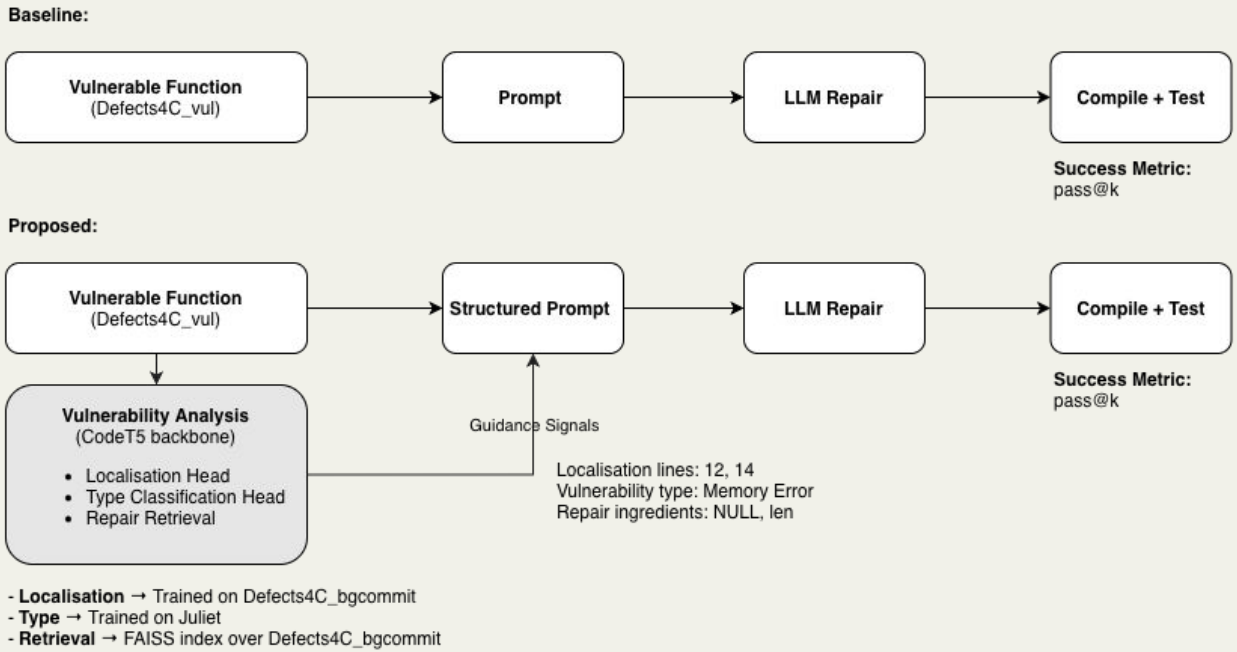
To what extent do *structured vulnerability analysis signals* improve LLM-based vulnerability repair?

(localisation, type classification, repair ingredients)

RESEARCH QUESTION 3 (RQ3)

How sensitive is repair performance to the *accuracy of vulnerability localisation*?

Proposed Repair Pipeline



Vulnerability Analysis Model: CodeT5

IDENTIFIER-AWARE PRETRAINING

CodeT5 introduces identifier-aware objectives that explicitly model variable and function names, improving understanding of program semantics [6,7].

CODE-AWARE PRETRAINING

Pretrained on *8.3M functions across 8 programming languages*, including C, making it suitable for vulnerability analysis and repair tasks.

UNIFIED UNDERSTANDING + GENERATION

Its encoder-decoder architecture supports both *code understanding tasks* (classification, defect detection) and *generation tasks* (code synthesis and repair).

Structured Analysis Signals

VULNERABILITY LOCALISATION

A *binary classification head* performs binary classification on each line if it's associated with the vulnerability. Training supervision is derived from prior *Defects4C_bgcommit* code commits, where modified regions provide supervision for potentially vulnerable locations.

VULNERABILITY TYPE CLASSIFIER

A *classification head* predicts the vulnerability category (Signature, Sanitizer, Memory Error, Logic Organization). The model is trained on the labeled C vulnerability dataset *Juliet* [5].

REPAIR INGREDIENT RETRIEVAL

Retrieval is implemented using FAISS over embeddings from the *CodeT5 encoder*. Similar prior fixes are retrieved and tokens introduced in those patches are extracted as candidate repair ingredients (e.g. NULL checks, length variables, bounds checks).

Automatic Generation of Localisation Labels

Pre-Commit Function:

```
1   void update_value(int *ptr)
2   {
3       int value = 10;
4
5       ...
6       *ptr = value;        ← suspicious
7       ...
8   }
```

Post-Commit Function:

```
1   void update_value(int *ptr)
2   {
3       int value = 10;
4
5       ...
6       if (ptr != NULL)
7           *ptr = value;
8       ...
9   }
```

Diff Extraction (Patch Commit):

```
- *ptr = value;
+ if (ptr != NULL)
+     *ptr = value;
```

Localisation Labels:

```
[6]
```

Line-level Label Vector: Labels derived from modified lines (weak supervision for localisation).

```
[0,0,0,0,0,1,0,0]
```

Training Dataset Entry:

```
{
  function: update_value(...),
  labels: [0,0,0,0,0,1,0,0]
}
```

Vulnerability Type Mapping (Juliet Dataset)

118 Juliet CWE types are mapped to the four vulnerability categories defined in Defects4C and used to train the vulnerability type classifier [3].

Structured Prompt

SIGNAL INTEGRATION

Localisation, vulnerability type, and repair ingredients are incorporated into the repair prompt.

STRUCTURED GUIDANCE

Signals are presented as explicit guidance alongside the vulnerable function.

CONDITIONED GENERATION

The LLM generates candidate patches conditioned on this structured context.

The following function contains a vulnerability:
```
[Original Vulnerable Function]
```
The error message from test case is:
[Error Message]

Localisation lines: 12, 14
Vulnerability type: Memory Error
Repair ingredients: NULL, len

Please repair the vulnerability in the function and tell me the complete repaired function.

[Original Vulnerable Function]* =
static inline int s_base64_get_decoded_value(char_to_decode, uint8_t*value, int8_t allow_sentinal) {
...
return AWS_OP_ERR;}

[Error Message]* =
FAILURE Expected error but no error occurred; rv=0, aws_last_error=0000 (expected 0007)

*Example adapted from Defects4C paper.

LLM Repair

PATCH GENERATION

LLMs generate candidate patches for *vulnerable functions from the Defects4C_vul benchmark*, conditioned on the structured prompt.

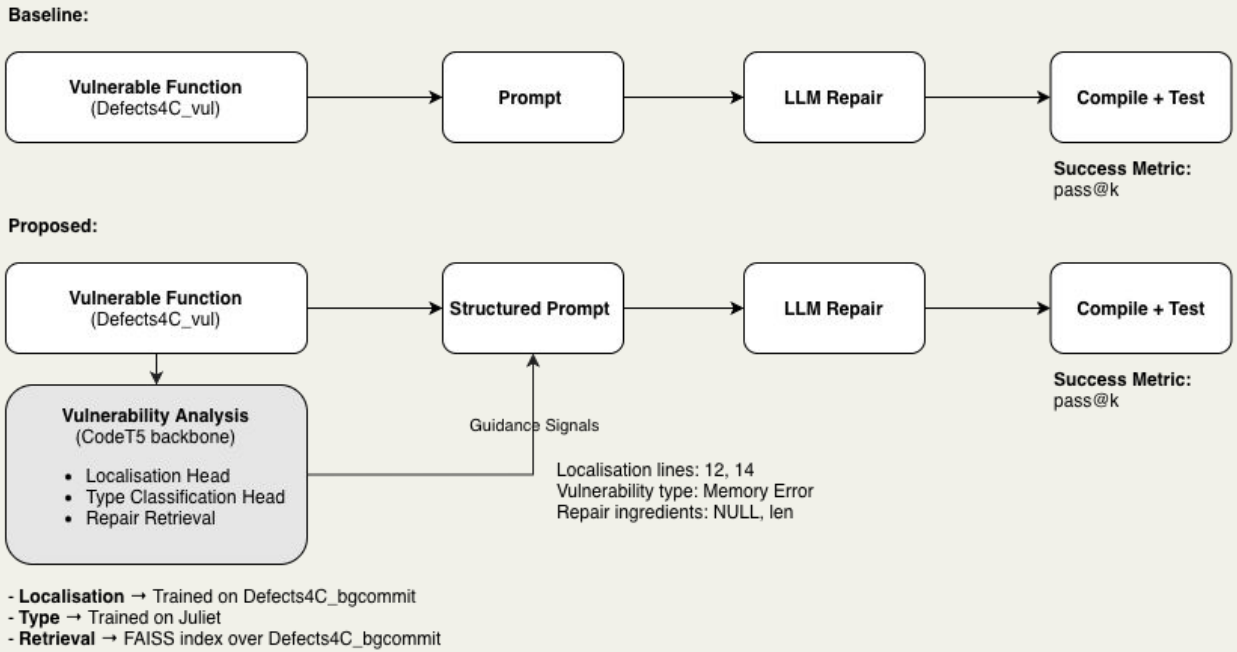
MULTIPLE CANDIDATE PATCHES

For each vulnerability, the LLM generates *top-k candidate patches*, which are later validated through compilation and testing.

MODEL VARIABILITY

Prior studies show repair performance varies significantly across LLMs, motivating evaluation across multiple LLM architectures [4].

Proposed Repair Pipeline



Experimental Setup

MODELS + DATASET

Evaluate multiple LLMs (e.g. LLaMA, DeepSeek, Qwen) on the Defects4C_vul dataset (102 vulnerabilities).

BASELINE

LLM repair without structured analysis signals. To evaluate localisation sensitivity, we compare predicted localisation with ground-truth localisation derived from patch diffs.

ABLATION STUDY

Evaluate the contribution of each analysis signal by testing all combinations of localisation (L), type classification (T), and repair ingredient retrieval (R).

Configurations evaluated: Baseline, L, T, R, L+T, L+R, T+R, L+T+R

Compile + Test

PATCH COMPILATION

Generated patches are applied to vulnerable functions from the *Defects4C_vul* benchmark. Patches that fail to compile are discarded before further evaluation.

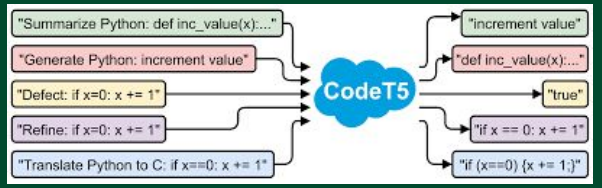
TEST SUITE VALIDATION

Compiled patches are evaluated against the *Defects4C_vul* test suite. A patch is considered *correct* if it compiles and passes all tests in the Defects4C_vul suite.

REPAIR SUCCESS (pass@k)

pass@k measures the probability that at least one correct patch appears among the top *k* generated candidates.

Thank You!



LLaMA by Meta

deepseek

Qwen

References

[1] Jiang, N., Lutellier, T., & Tan, L. (2021). CURE: Code-Aware Neural Machine Translation for Automatic Program Repair. *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 1161-1173.
[2] Huang, Z., Lie, D., Tan, G., & Jaeger, T. (2019). Using Safety Properties to Generate Vulnerability Patches. *2019 IEEE Symposium on Security and Privacy (SP)*, 539-554.
[3] Wang, J., Xie, X., Hu, Q., Liu, S., Yu, J., Klong, J., & Li, Y. (2025). Defects4C: Benchmarking Large Language Model Repair Capability with C/C++ Bugs. *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 254-265.
[4] Campos, V., Shariffdeen, R., Ulges, A., & Noller, Y. (2025). Exploring Generalizable Automated Program Repair with Large Language Models.
[5] Black, P. E., & Black, P. E. (2018). *Juliet 1.3 test suite: Changes from 1.2*. Gaithersburg, MD, USA: US Department of Commerce, National Institute of Standards and Technology.
[6] Wang, Y., Wang, W., Joty, S.R., & Hoi, S.C. (2021). CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. *ArXiv, abs/2109.00859*.
[7] Wang, W., Wang, Y., Joty, S.R., & Hoi, S.C. (2023). RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.